Phase 1: Agile Process — Scrum + XP Security Practices

Course Code: 20CYS401 | **Project**: Online Book Review & Recommendation Platform

---

1.1 Agile Methodology Selection

Chosen Framework: **Scrum + Extreme Programming (XP) Security Practices**

Rationale

| Factor | Justification |

|---|---|

| Short iterative sprints | Allows early security verification before features accumulate |

| Continuous integration | Enables automated security scanning in every sprint |

| Test-first practices | TDD is natural for security authorization tests (IDOR, RBAC) |

| Refactoring discipline | XP's refactoring culture keeps code auditable and secure |

| Pair review | Simulates peer code review as part of Developer Sprint activities |

1.2 Mapping of At Least 5 Agile Principles to Project

| # | Agile Principle | Application | Security Benefit | Evidence |

|---|---|---|---|---|

| P-01 | Working software over comprehensive documentation | Flask application with all 12 REQs implemented before documentation written | Ensures security claims are grounded in real working implementation | `app/` directory with full backend |

| P-02 | Welcome changing requirements | IDOR was discovered mid-development; authorization checks restructured across all routes | Security weaknesses (VUL-01) fixed iteratively without project restart | `app/services/review_service.py` CTRL-02 |

| P-03 | Continuous attention to technical excellence | SQLAlchemy ORM used throughout (zero string-concatenated queries); Bcrypt rounds=12; HttpOnly sessions | Eliminates entire categories of injection and session-hijacking attacks | `app/core/security.py`, `app/config.py` |

| P-04 | Simplicity — maximize work not done | Jinja2 server-rendered templates instead of SPA framework; SQLite instead of cloud DB | Smaller attack surface; fewer external dependencies; no external API keys | `app/templates/`, `requirements.txt` |

| P-05 | At regular intervals, reflect and adapt | Sprint 1 retrospective identified: absence of audit logging; immediately added COMP-08 before Sprint 2 | Audit logging (CTRL-08) added before any admin features went into production | `app/core/audit.py` |

1.3 Scrum Process Applied

Sprint Cycle (2 Sprints × 1 week)

```
Sprint 1 → Sprint 2
Core Auth & Catalog → Reviews, Reading List, Admin, Security Hardening
```

Artifacts

- **Product Backlog**: 10 User Stories, see `docs/phase-09-backlog/`

- **Sprint Backlog**: TASK-01 through TASK-13

- **Sprint Review**: Functional increment demonstrated to product owner
- **Sprint Retrospective**: Documented below

1.4 XP Security Practices Implemented

Practice	Implementation
Test-First Development	`test_authorization.py` written defining ownership constraints *before* service layer
Continuous Integration	GitHub Actions `.github/workflows/ci.yml` runs tests + Bandit on every push
Refactoring	Two verified refactorings documented in Section 1.5
Coding Standards	Consistent service/route/model layered structure across all components
Simple Design	Minimal dependencies; no unnecessary complexity

---

1.5 Two Verified Refactorings (from Git History)

Refactoring 1 — Monolithic Route to Layered Service Architecture

Stage	Description
Before	All business logic inside Flask route functions in routes/reviews.py (create, validate, persist)
Problem	No separation of concerns; impossible to unit-test authorization logic in isolation
Refactoring	Extracted `ReviewService`, `UserService`, `BookService`, `ReadingListService` classes
After	Routes become thin controllers: only HTTP handling; all business logic in services
Security Improvement	Object-level authorization checks (`is_owned_by()`) now centralized in `ReviewService.update_review()` and tested independently
Evidence	`app/services/review_service.py`, `app/routes/reviews.py`

Refactoring 2 — Inline SQL String Replacement with Parameterized ORM Queries

Stage	Description
Before	Early prototype of search used: `db.session.execute(f"SELECT * FROM books WHERE title LIKE '%{query}%'")`
Problem	Direct string interpolation into SQL expression — classic SQL injection vulnerability (VUL-06)
Refactoring	Replaced with `Book.query.filter(Book.title.ilike(f"%{cleaned_query}%"))` via SQLAlchemy ORM
After	All search queries bound as ORM parameters; no raw string SQL anywhere in codebase
Security Improvement	Eliminates SQL injection attack surface as proven by TEST-03 (`test_search_sql_injection_metacharacters PASSED`)
Evidence	`app/services/search_service.py`, `tests/test_unit_search.py`

1.6 Agile Security Risks and Mitigations

| Risk | Impact | Mitigation |

|---|---|---|

| **Risk 1: Shipping untested authorization** — Sprint pressure to deliver features could cause authorization checks to be deferred | High (IDOR exposure for all review data) | Sprint Definition of Done mandates that every PR touching review mutations must have a corresponding authorization test in `test_authorization.py` before merge |

| **Risk 2: Credential creep in test/seed code** — Demonstration passwords in `scripts/seed.py` might leak to production | Medium (Brute-force or reuse risk) | `.gitignore` excludes `.env`; seed passwords are documented as "local demonstration only"; Bandit scan flags hardcoded test credentials as Low/CWE-259 |